ZÁRÓDOLGOZAT

Készítették:

Bódi Szabolcs – Oravecz Ádám József - Tátrai Levente

Konzulens:

Németh Bence

Miskolc

2026.

Miskolci SZC Kandó Kálmán Informatikai Technikum

Miskolci Szakképzési Centrum

SZOFTVERFEJLESZTŐ- ÉS TESZTELŐ SZAK

Dokumentáció

Arcade Mania

Bódi Szabolcs – Oravecz Ádám József - Tátrai Levente

2025-2026

Tartalomjegyzék

A projekt bemutatása **1**
Bevezetés 1
A projekt célja 2
Technológiai háttér 2
Fejlesztési környezet 3
A projekt felépítése 4
Adatbázis **6**
Az adatbázis felépítése 6
Backend **7**
A backend rendszer architektúrája 7
A backend rendszer architektúrája 8
Pontszám rendszer 9
Backend és Frontend kommunikáció 10
API végpontok működése 11
Hibakezelés és validáció 12
WPF kliens 12
Frontend **14**
A frontend alkalmazás architektúrája 14
Oldalak és navigáció 15
Autentikáció kezelése a frontendben 16
Komponens alapú felépítése 17
Játékok implementációja 18
API kommunikáció a frontendben 19
Állapotkezelés és állapotlogika 19
Tesztelés **21**
A projekt komponenseinek tesztelése 21
Összegzés **22**
A projekt összegzése 22
Irodalomjegyzék **23**
Melléklet ..... **I**
Adatbázis modell: II
Frontend komponens példa (RequireAuth): II
Backend API példa (Login): II

A projekt bemutatása

Bevezetés

Az Arcade Mania projekt célja egy online arcade játékplatform létrehozása – az „y8” névre hallgató online játékplatform mintájára – amelyben a felhasználók különböző egyszerű, klasszikus stílusú játékokat játszhatnak, miközben a rendszer rögzíti és rangsorolja az elért eredményeiket. A projekt egy teljes értékű webes alkalmazásként valósul meg, amely frontend, backend és adatbázis komponensekből épül fel.

A fejlesztés során fontos szempont volt, hogy a rendszer ne csupán működőképes legyen, hanem olyan technológiákat alkalmazzon, amelyek a modern webfejlesztésben is széles körben használatosak. Ennek megfelelően a frontend React alapokon készült, míg a backend ASP.NET Core Web API technológiával lett megvalósítva. Az adatok tárolására MySQL adatbázis szolgál. A projekt admin felületét WPF asztali alkalmazásként készítettük el C# nyelven, hasonlóan az ASP.NET Core Web API-hoz.

A projekt egyben tanulási célokat is szolgál: lehetőséget ad arra, hogy a fejlesztés során a csapat tagjai gyakorlati tapasztalatot szerezzenek több különböző technológia integrálásában. A rendszer fejlesztése során külön figyelmet fordítottunk az alkalmazás architektúrájára, a backend és frontend közötti kommunikációra, valamint az adatbiztonságra.

Az Arcade Mania egy olyan platform, amely a klasszikus játéktermek hangulatát próbálja modern webes környezetben megjeleníteni. A játékok egyszerű mechanikákra épülnek, azonban a rendszer mögött egy komplex infrastruktúra működik, amely kezeli a felhasználókat, és az autentikációt.

A projekt során a teljes backend-et Bódi Szabolcs, a frontend-et szinte teljesne Oravecz Ádám József, az adatbázist és a vele kapcsolatos dolgokat, valamint a design-t pedig Tátrai Levente fejlesztette. Ezenfelül Levente részt vett még a frontend fejlesztésében is Ádám mellett, ő készítette a memória játékot.

A projekt célja

A projekt célja egy olyan online játékplatform létrehozása volt, amelyben a felhasználók regisztráció után különböző arcade jellegű játékokat játszhatnak, és az elért pontszámaik automatikusan rögzítésre kerülnek az adatbázisban.

A rendszer alapvető funkciói a következők:

- felhasználói regisztráció és bejelentkezés
- játékok indítása webes felületen
- pontszámok rögzítése és tárolása
- ranglisták megjelenítése
- felhasználói profil kezelése

A platform célja nem egy komplex videojáték létrehozása volt, hanem egy olyan rendszer fejlesztése, amely bemutatja egy modern webalkalmazás teljes architektúráját. Ennek érdekében a projekt tartalmaz frontend felületet, REST API alapú backend rendszert, valamint relációs adatbázist és a projekthez tartozó admin panelt.

A projekt egyik fontos célja az volt, hogy a fejlesztés során különböző technológiai megoldásokat integráljunk egyetlen rendszerbe. Ennek köszönhetően a platform nemcsak játékok futtatására alkalmas, hanem bemutatja a kliens és szerver közötti kommunikáció működését, valamint az adatkezelés különböző aspektusait is.

Technológiai háttér

A rendszer több különböző technológia együttműködésével valósult meg. A fejlesztés során a cél az volt, hogy olyan eszközöket használjunk, amelyek a modern webfejlesztésben is elterjedtek, és lehetővé teszik egy stabil és skálázható rendszer kialakítását.

A frontend rész React keretrendszer segítségével készült. A React egy komponens alapú JavaScript könyvtár, amely lehetővé teszi dinamikus felhasználói felületek létrehozását. A React használatával az alkalmazás egy Single Page Application (SPA) formájában működik, ami azt jelenti, hogy az oldalak közötti navigáció teljes újratöltés nélkül történik.

A frontend fejlesztéséhez a Vite build rendszer került alkalmazásra. A Vite gyors fejlesztési környezetet biztosít, amely lehetővé teszi a fejlesztés során a valós idejű frissítést és a gyors build folyamatot.

A backend rész ASP.NET Core Web API technológiával készült. Ez a keretrendszer lehetővé teszi REST alapú szolgáltatások létrehozását, amelyek HTTP protokollon keresztül kommunikálnak a frontend alkalmazással. A backend feladata többek között a felhasználók kezelése, az autentikáció biztosítása, valamint a játékok pontszámainak rögzítése.

Az adatbázis kezelésére a MySQL adatbázis-kezelő rendszert használunk. A backend és az adatbázis közötti kapcsolatot az Entity Framework Core ORM réteg biztosítja, amely lehetővé teszi, hogy az adatbázis műveletek objektumorientált módon történjenek.

A fejlesztés során használt fő technológiák tehát a következők:

- React
- Vite
- JavaScript
- ASP.NET Core Web API
- C# WPF
- Entity Framework Core
- MySQL

Ezek a technológiák együtt egy olyan teljes stack alkalmazást alkotnak, amelyben a kliens és szerver oldali komponensek egymással együttműködve biztosítják a rendszer működését.

Fejlesztési környezet

A projekt fejlesztése során több különböző fejlesztői környezetet használtunk annak érdekében, hogy a különböző technológiai komponensek hatékonyan fejleszthetők legyenek.

A frontend fejlesztéséhez a Visual Studio Code nevű fejlesztői környezetet használtuk. Ez egy könnyű, gyors és rugalmas fejlesztői környezet, amely számos bővítmény segítségével

támogatja a JavaScript és React alapú fejlesztést. A VSC egyik előnye, hogy jól integrálható különböző build eszközökkel és csomagkezelőkkel, például a bun rendszerrel. A fejlesztés során több extension-t is használtunk, mint például a „ES7+ React/Redux/React-Native snippets” vagy a „Python Debugger”.

A backend fejlesztése Visual Studio fejlesztői környezetben történt. A Visual Studio különösen jól támogatja a C# nyelven történő fejlesztést, valamint az ASP.NET Core alkalmazások létrehozását. A környezet lehetővé teszi az API endpointok tesztelését, a hibakeresést, valamint az adatbázis kapcsolatok kezelését is. A backend fejlesztése során olyan NuGet Package-eket használtunk fel, mint például az „MSTest”, „MySql.EntityFramework.Core”, „Microsoft.EntityFrameworkCore.Tools” vagy a „NETCore.Encrypt”

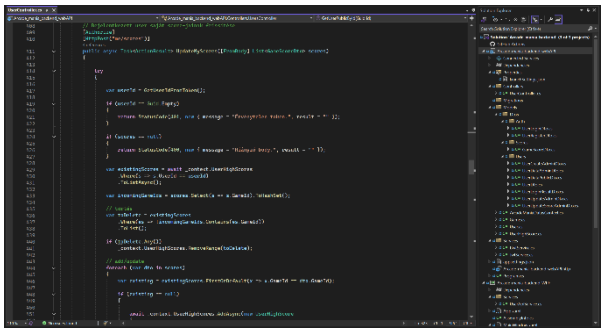
Az adatbázis futtatására és tesztelésére a XAMPP környezetet használtuk, amely tartalmazza a MySQL adatbázis-kezelőt is. Ennek segítségével a teljes rendszer lokálisan futtatható volt a fejlesztési folyamat során.

A fejlesztés során verziókezelésre Git rendszert használtunk, a kód tárolására pedig GitHub repository szolgált. A verziókezelés lehetővé tette, hogy a csapat tagjai párhuzamosan dolgozzanak a projekt különböző részein, miközben a változtatások biztonságosan összevonhatók voltak.

A projekt felépítése

Az Arcade Mania rendszer több egymással együttműködő komponensből áll. A teljes alkalmazás három fő részből épül fel:

- frontend alkalmazás
- backend API és admin WPF (1.ábra)
- adatbázis



1. ábra – Backend API végpont

A frontend egy React alapú webalkalmazás, amely a felhasználó számára biztosítja a grafikus felületet. A felhasználók ezen keresztül tudnak regisztrálni, bejelentkezni, játékokat indítani, valamint megtekinteni a ranglistákat.

A backend egy ASP.NET Core Web API alapú szerveralkalmazás, amely a frontend által küldött HTTP kéréseket dolgozza fel. A backend felelős az autentikációért, a felhasználók kezeléséért, valamint az adatbázis műveletek végrehajtásáért.

A WPF admin panel a fejlesztők és az üzemeltetők számra készült, ahol egy helyen lehet nyomon követni az összes user felhasználói adatait és pontszámait. Természetesen az admin felületen ezen adatok módosítása lehetséges, ami igen hasznos és megkerülhetetlen eszközzé teszi.

Az adatbázis relációs modellben tárolja a felhasználók adatait, a játékokat és a pontszámokat. Az adatbázis biztosítja, hogy a játékok eredményei tartósan megmaradjanak, valamint lehetővé teszi a ranglisták generálását.

A rendszer működése során a frontend HTTP kéréseket küld a backend API számára. A backend feldolgozza ezeket a kéréseket, kommunikál az adatbázissal, majd visszaküldi a szükséges adatokat a kliensnek. Ennek a kliens-szerver kommunikációnak köszönhetően a rendszer dinamikusan működik, és a felhasználók valós időben láthatják az adatok változásait.

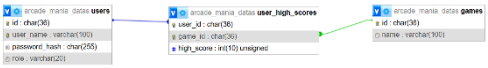
A projekt struktúrája moduláris felépítésű, ami azt jelenti, hogy a különböző komponensek egymástól elkülönítve, mégis együttműködve működnek. Ez a megközelítés megkönnyíti a rendszer karbantartását és a későbbi bővítéseket.

Adatbázis

Az adatbázis felépítése

Az Arcade Mania rendszer adatainak tárolására relációs adatbázist alkalmaztunk. A projektben MySQL adatbázis-kezelő rendszert használunk, amely stabil és széles körben elterjedt megoldás webalkalmazások adatainak kezelésére.

Az adatbázis neve „arcade_mania_datas” (2.ábra), amely három fő táblát tartalmaz. Ezek a táblák a felhasználók adatait, a játékokat és a játékokban elért pontszámokat tárolják. A „users” tábla tartalmazza a rendszer felhasználóinak adatait. A tábla elsődleges kulcsa egy GUID típusú azonosító, amely egyedi módon azonosít minden felhasználót. A felhasználókhoz tartozó adatok közé tartozik a felhasználónév, a jelszó titkosított formája, valamint a felhasználói szerepkör. A szerepkör lehet például „User” vagy „Admin”, amely a rendszerben elérhető jogosultságokat határozza meg.



2. ábra – Az adatbázis felépítése

A „games” tábla tartalmazza a rendszerben elérhető játékokat. Minden játék egy egyedi azonosítóval és egy név mezővel rendelkezik. A rendszerben jelenleg három játék szerepel: Fighter, Memory és Snake. Ezek az azonosítók fix értékek, amelyek a backend és frontend alkalmazások számára is hivatkozási pontként szolgálnak.

A felhasználók és a játékok közötti kapcsolatot a „user_high_scores” kapcsolótábla biztosítja. Ez a tábla minden felhasználóhoz és játékhoz eltárolja az adott játékban elért legmagasabb pontszámot. A tábla összetett elsődleges kulccsal rendelkezik, amely a felhasználó azonosítójából és a játék azonosítójából áll. Ez biztosítja, hogy egy felhasználó minden játékhoz legfeljebb egy rekorddal rendelkezzen.

A kapcsolatok idegen kulcsok segítségével kerültek definiálásra. A user_high_scores.user_id mező a users tábla azonosítójára hivatkozik, míg a game_id mező a games tábla rekordjára mutat. A rendszer cascade törlést alkalmaz, amely biztosítja, hogy egy felhasználó törlése esetén a hozzá tartozó pontszámok automatikusan eltávolításra kerüljenek.

Backend

A backend rendszer architektúrája

Az Arcade Mania backend része ASP.NET Core Web API technológiával készült. A Web API egy szerveroldali alkalmazás, amely HTTP protokollon keresztül kommunikál a frontend alkalmazással.

A backend elsődleges feladata a kliens kéréseinek feldolgozása, a felhasználók hitelesítése, valamint az adatbázis műveletek végrehajtása. A rendszer REST alapú architektúrát követ, amelyben az egyes funkciók külön API végpontokon keresztül érhetők el.

A backend projekt moduláris felépítésű, és több különböző rétegből áll. A legfontosabb komponensek a következők:

- Controllers
- Models
- DTO-k
- Services
- Adatbázis réteg

A Controllers réteg kezeli az API végpontokat. A frontend alkalmazás HTTP kéréseket küld ezekhez a végpontokhoz, amelyeket a controller osztályok fogadnak és feldolgoznak. A controller réteg felelős például a bejelentkezési és regisztrációs folyamatok kezeléséért.

A Models réteg tartalmazza azokat az objektumokat, amelyek az adatbázis táblák leképezésére szolgálnak. Ezek az objektumok reprezentálják a felhasználókat, a játékokat és a pontszámokat.

A DTO (Data Transfer Object) osztályok az API kommunikáció során használt adatstruktúrákat definiálják. A DTO-k célja, hogy elkülönítsék a belső adatmodelleket az API által küldött vagy fogadott adatoktól. Ez növeli a biztonságot és átláthatóbbá teszi az adatkezelést.

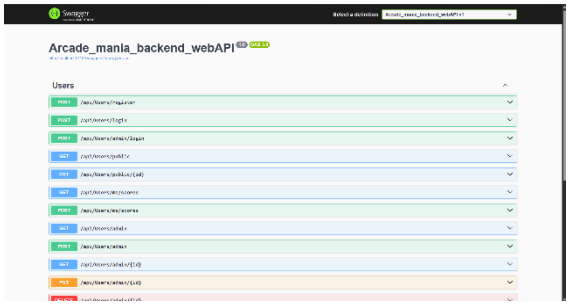
A Services réteg tartalmazza az üzleti logikát. Ide tartoznak például az autentikációval kapcsolatos műveletek vagy a pontszámok feldolgozása. Ez a réteg biztosítja, hogy a controller osztályok ne tartalmazzanak túl sok logikai feldolgozást, így a kód könnyebben karbantartható marad.

A backend és az adatbázis közötti kapcsolatot az Entity Framework Core ORM réteg biztosítja. Ez lehetővé teszi, hogy az adatbázis műveletek objektumorientált módon történjenek.

A backend rendszer architektúrája

Az Arcade Mania backend rendszere ASP.NET Core Web API technológiával készült. A Web API egy szerveroldali alkalmazás, amely HTTP protokollon keresztül kommunikál a frontend alkalmazással, és különböző szolgáltatásokat biztosít a kliens számára.

A backend egyik legfontosabb feladata a kliens kéréseinek feldolgozása (3.ábra). Ide tartozik például a felhasználók regisztrációja, a bejelentkezési folyamat kezelése, a játékok pontszámainak rögzítése, valamint a ranglisták lekérdezése. A rendszer REST alapú architektúrát követ, amelyben az egyes funkciók külön API végpontokon keresztül érhetők el.



3. ábra – .NET API swagger

A backend projekt réteges architektúrát alkalmaz. Ez azt jelenti, hogy az alkalmazás külön logikai rétegekre van bontva, amelyek jól elkülöníthetők egymástól. A legfontosabb rétegek a következők:

- Controller réteg
- Service réteg
- Adatmodell réteg
- Adatbázis réteg

A Controller réteg felelős az API végpontok kezeléséért. Amikor a frontend egy HTTP kérést küld a szervernek, a kérés először a megfelelő controller osztályhoz érkezik. A controller feldolgozza a bemeneti adatokat, majd meghívja a szükséges szolgáltatásokat.

A Service réteg tartalmazza az alkalmazás üzleti logikáját. Ez a réteg végzi azokat a műveleteket, amelyek az alkalmazás működéséhez szükségesek, például a felhasználók

hitelesítését vagy a pontszámok feldolgozását. A szolgáltatások használata segít abban, hogy a controller osztályok egyszerűbbek és áttekinthetőbbek maradjanak.

Az adatmodell réteg tartalmazza azokat az objektumokat, amelyek az adatbázis táblákat reprezentálják. Ezek az objektumok például a felhasználókat, a játékokat és a pontszámokat reprezentálják a backend alkalmazásban.

Az adatbázissal való kommunikációt az Entity Framework Core ORM réteg biztosítja. Ez az eszköz lehetővé teszi, hogy az adatbázis műveletek objektumorientált módon történjenek, így a fejlesztőknek nem szükséges minden lekérdezést manuálisan SQL-ben megírni.

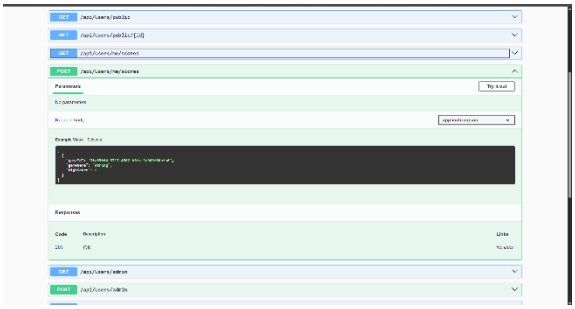
Az Entity Framework egyik előnye, hogy automatikusan kezeli az objektumok és az adatbázis táblák közötti kapcsolatot. Ez jelentősen megkönnyíti a fejlesztést, és csökkenti az adatkezeléssel kapcsolatos hibák lehetőségét.

A backend architektúra egyik fontos tulajdonsága a modularitás. A rendszer különálló komponensekből épül fel, amelyek egymástól függetlenül fejleszthetők és módosíthatók. Ez a megközelítés megkönnyíti a rendszer karbantartását, valamint lehetővé teszi a későbbi bővítéseket is.

Pontszám rendszer

Az Arcade Mania rendszer egyik legfontosabb funkciója a játékok során elért pontszámok kezelése. A platform célja nem csupán az, hogy a felhasználók játszhassanak a játékokkal, hanem az is, hogy az elért eredmények rögzítésre kerüljenek, és ezek alapján ranglisták készülhessenek.

A pontszámok az adatbázis user_high_scores táblájában kerülnek tárolásra (4.ábra). Ez a tábla minden felhasználóhoz és minden játékhoz egy rekordot tartalmaz. A rekord a felhasználó által elért legmagasabb pontszámot rögzíti.



4. ábra – .NET API "me/score" post végpont

A pontszámok kezelése a backend egyik fontos feladata. Amikor egy játék véget ér, a frontend elküldi az elért pontszámot a backend API számára. A backend ezt követően ellenőrzi, hogy az adott felhasználó számára már létezik-e rekord az adott játékhoz.

Ha a felhasználó korábban még nem játszott az adott játékkal, akkor a rendszer létrehoz egy új rekordot az adatbázisban. Amennyiben már létezik rekord, a backend összehasonlítja az új pontszámot a korábbi értékkel.

Ha az új pontszám magasabb a korábbi rekordnál, a rendszer frissíti az adatbázisban tárolt értéket. Ha az új pontszám alacsonyabb, akkor a rekord változatlan marad. Ez a mechanizmus biztosítja, hogy a rendszer mindig a felhasználó legjobb eredményét tárolja.

A ranglisták generálása az adatbázis lekérdezése alapján történik. A backend a játékhoz tartozó pontszámokat csökkenő sorrendben rendezi, majd visszaküldi az adatokat a frontend alkalmazás számára. A frontend ezt követően megjeleníti a ranglistát a felhasználói felületen.

A ranglista rendszer fontos szerepet játszik a felhasználói élményben. A játékosok összehasonlíthatják az eredményeiket más felhasználókkal, ami növeli a platform versenyképességét és motivációt ad a jobb eredmények elérésére.

A pontszám rendszer tervezése során fontos szempont volt az adatkonzisztencia és a teljesítmény. Az adatbázis struktúrája lehetővé teszi, hogy a ranglisták gyorsan lekérdezhetők legyenek, még nagyobb mennyiségű adat esetén is.

Backend és Frontend kommunikáció

A frontend és backend közötti kommunikáció HTTP alapú REST API segítségével történik. A frontend alkalmazás különböző API hívásokat küld a backend szervernek, amelyek JSON formátumban továbbítják az adatokat.

A legfontosabb API műveletek közé tartozik:

- felhasználói regisztráció
- bejelentkezés
- felhasználói adatok lekérése
- pontszámok lekérése
- pontszámok frissítése

Az API válaszok JSON formátumban kerülnek visszaküldésre a kliens számára. A frontend alkalmazás ezeket az adatokat feldolgozza, majd megjeleníti a felhasználói felületen.

Ez a kliens–szerver kommunikáció biztosítja, hogy az alkalmazás dinamikusan működjön, és a felhasználók valós időben láthassák az adatok változásait.

API végpontok működése

A backend rendszer szolgáltatásai REST alapú API végpontokon keresztül érhetők el. Ezek a végpontok HTTP kérések segítségével kommunikálnak a kliens alkalmazásokkal.

Az API végpontok különböző típusú műveleteket támogatnak, például adatlekérdezést, új adatok létrehozását vagy meglévő adatok módosítását. A kommunikáció során az adatok JSON formátumban kerülnek továbbításra.

A legfontosabb API műveletek közé tartoznak:

- felhasználó regisztráció
- felhasználó bejelentkezés
- felhasználói adatok lekérdezése
- pontszámok lekérdezése
- pontszámok frissítése

Az API végpontok kialakítása során fontos szempont volt az átláthatóság és a konzisztens működés. Minden végpont egy jól meghatározott funkcióért felel, és a válaszok egységes formátumban kerülnek visszaküldésre.

Ez a struktúra lehetővé teszi, hogy a backend szolgáltatásai más alkalmazásokból is könnyen elérhetők legyenek, például a webes frontend vagy a WPF kliens alkalmazás számára.

Hibakezelés és validáció

Egy webalkalmazás működése során fontos, hogy a rendszer megfelelően kezelje a hibás vagy hiányos adatokat. A backend rendszer ezért külön figyelmet fordít az adatvalidációra és a hibakezelésre.

Az API végpontok ellenőrzik a beérkező adatokat, például a felhasználónév és jelszó mezőket a bejelentkezési folyamat során. Ha a szükséges adatok hiányoznak vagy érvénytelenek, a rendszer megfelelő hibaüzenetet küld vissza a kliens számára.

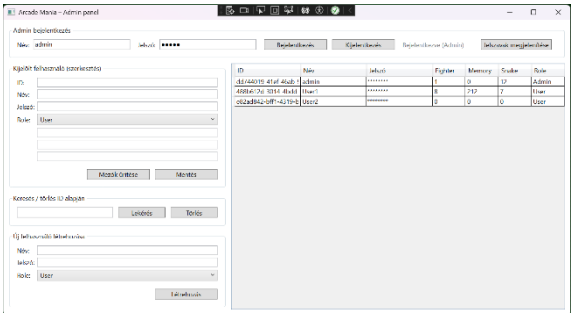
A hibakezelés célja az is, hogy a rendszer stabil maradjon váratlan helyzetek esetén is. Például ha az adatbázis kapcsolat megszakad, vagy a kliens hibás adatot küld, a backend megfelelő válaszkóddal jelzi a problémát.

Az ilyen jellegű ellenőrzések növelik az alkalmazás megbízhatóságát, és segítenek abban, hogy a rendszer hosszú távon is stabilan működjön.

WPF kliens

A projekt részeként egy WPF alapú kliens alkalmazás is készült (5.ábra). Ez az alkalmazás a backend API-val kommunikál, és elsősorban adminisztrációs vagy tesztelési célokra használható.

A WPF alkalmazás C# nyelven készült, és a Windows Presentation Foundation keretrendszert használja a grafikus felhasználói felület megvalósításához.



5. ábra – Backend WPF admin felület

Az alkalmazás egyik fontos komponense a UserApiService, amely a backend API-val való kommunikációért felel. Ez a szolgáltatás HTTP kéréseket küld a Web API számára, majd feldolgozza a visszakapott válaszokat.

A WPF kliens lehetővé teszi például a felhasználói adatok lekérdezését vagy módosítását anélkül, hogy a webes frontend alkalmazást kellene használni. Ez különösen hasznos lehet a rendszer tesztelése vagy adminisztrációja során.

A WPF alkalmazás működése jól demonstrálja, hogy a backend API nem kizárólag a webes frontend számára érhető el, hanem más kliens alkalmazások is képesek csatlakozni hozzá.

Ez a megközelítés jól illusztrálja a REST API egyik legfontosabb előnyét: a szerver oldali szolgáltatások platformfüggetlen módon használhatók különböző kliens alkalmazásokból.

Frontend

A frontend alkalmazás architektúrája

Az Arcade Mania rendszer frontend része egy React alapú webalkalmazás, amely a felhasználók számára biztosítja a grafikus felületet. A frontend feladata, hogy lehetővé tegye a felhasználók számára a rendszer funkcióinak elérését, például a regisztrációt, a bejelentkezést, a játékok indítását és a ranglisták megtekintését.

A frontend egy Single Page Application (SPA) architektúrát követ. Ez azt jelenti, hogy az alkalmazás egyetlen HTML oldalból indul, és a különböző nézetek között JavaScript segítségével történik a navigáció. Ennek előnye, hogy az oldalak közötti váltás gyorsabb, mivel nem szükséges minden alkalommal teljes oldalt újratölteni.

A projekt a React könyvtárat használja a felhasználói felület megvalósításához. A React egyik legfontosabb jellemzője a komponens alapú felépítés. Az alkalmazás különálló komponensekből áll, amelyek egymástól függetlenül fejleszthetők és újra felhasználhatók.

A frontend fejlesztéséhez a Vite build rendszer került alkalmazásra. A Vite gyors fejlesztési környezetet biztosít, amely lehetővé teszi a valós idejű frissítést és a gyors build folyamatot. Ez különösen hasznos a fejlesztési folyamat során, mivel az alkalmazás módosításai azonnal megjelennek a böngészőben.

Az alkalmazás forráskódja több logikai mappába szervezve található. A legfontosabb struktúra a következő:

- pages
- components
- auth
- services
- Games

Ez a struktúra segíti a projekt áttekinthetőségét, valamint megkönnyíti az egyes funkciók elkülönítését.

Oldalak és navigáció

A frontend alkalmazás több különböző oldalt tartalmaz, amelyek a React Router segítségével érhetők el. A React Router egy kliensoldali routing könyvtár, amely lehetővé teszi az alkalmazás különböző nézetei közötti navigációt.

Az alkalmazás fő oldalai a következők:

- Main oldal (6.ábra)
- Login oldal
- Register oldal
- Profile oldal
- Leaderboard oldal

6. ábra – Frontend main oldal

A Main oldal a rendszer központi felülete, ahol a felhasználók kiválaszthatják a játszható játékokat. Innen lehet elindítani a különböző játékokat.

A Login oldal a felhasználók bejelentkezésére szolgál. A felhasználók itt adhatják meg a felhasználónevüket és jelszavukat. A sikeres bejelentkezés után a rendszer egy autentikációs tokent generál, amelyet a frontend tárol.

A Register oldal lehetővé teszi új felhasználók regisztrációját. A felhasználók itt hozhatják létre saját fiókjukat, amely később a játékokhoz és ranglistákhoz kapcsolódik.

A Profile oldal a felhasználói adatok megjelenítésére szolgál. Itt láthatók például a felhasználó saját pontszámai.

A Leaderboard oldal a ranglisták megjelenítésére szolgál. A rendszer itt jeleníti meg az adott játék legjobb pontszámait.

A navigációt egy központi Header komponens biztosítja, amely minden oldalon megjelenik. Ez a komponens tartalmazza a menüt, valamint a navigációhoz szükséges gombokat.

Autentikáció kezelése a frontendben

A frontend alkalmazás egyik legfontosabb feladata a felhasználók hitelesítésének kezelése. Az Arcade Mania rendszer JWT (JSON Web Token) alapú autentikációt használ, amely lehetővé teszi a biztonságos kommunikációt a frontend és a backend között.

A hitelesítés kezelése a frontend alkalmazásban az AuthProvider nevű React komponens segítségével történik. Ez a komponens egy globális kontextust biztosít az alkalmazás számára, amely tartalmazza a bejelentkezett felhasználó adatait, valamint a bejelentkezéshez és kijelentkezéshez szükséges műveleteket.

A React Context használata lehetővé teszi, hogy az alkalmazás különböző komponensei hozzáférjenek az autentikációs adatokhoz anélkül, hogy azokat minden komponensnek külön át kellene adni. Ez jelentősen egyszerűsíti az alkalmazás felépítését, és biztosítja az állapotkezelés egységességét.

Amikor egy felhasználó sikeresen bejelentkezik, a backend szerver egy JWT tokent generál és visszaküldi a frontend számára. Ez a token tartalmazza a felhasználó azonosításához szükséges információkat, például a felhasználó azonosítóját és a token lejárati idejét. A frontend ezt a tokent eltárolja, majd minden további API kérés során elküldi a backend szervernek.

A felhasználói állapot tárolása a böngésző localStorage tárhelyében történik. Ennek segítségével a felhasználó bejelentkezési állapota akkor is megmarad, ha az oldal újratöltésre kerül. Amikor az alkalmazás elindul, az AuthProvider ellenőrzi, hogy létezik-e korábban eltárolt felhasználói adat. Ha igen, akkor az alkalmazás automatikusan visszaállítja a felhasználói állapotot.

Az AuthProvider rendszeresen ellenőrzi a token érvényességét is. Ha a token lejárt, a rendszer automatikusan kijelentkezteti a felhasználót, törli az eltárolt adatokat, és visszairányítja a felhasználót a bejelentkezési oldalra. Ez a mechanizmus biztosítja, hogy a rendszer csak érvényes hitelesítéssel legyen használható.

A frontend tartalmaz egy RequireAuth nevű komponenst is, amely a védett oldalak hozzáférését szabályozza. Ez a komponens ellenőrzi, hogy a felhasználó rendelkezik-e érvényes autentikációval. Ha nem, akkor a rendszer automatikusan átirányítja a felhasználót a bejelentkezési oldalra.

Ez a megoldás biztosítja, hogy a felhasználók csak hitelesítés után férjenek hozzá a rendszer bizonyos funkcióihoz, például a profiloldalhoz vagy a ranglistákhoz.

Az autentikáció frontend oldali kezelése fontos szerepet játszik a felhasználói élményben is. A megfelelő állapotkezelés biztosítja, hogy a felhasználó mindig a számára releváns felületet lássa, valamint, hogy a rendszer biztonságosan és stabilan működjön.

Összességében a JWT alapú autentikáció és a React Context használata egy modern és hatékony megoldást biztosít a felhasználói hitelesítés kezelésére a frontend alkalmazásban.

Komponens alapú felépítése

A React alkalmazás egyik legfontosabb előnye a komponens alapú architektúra. Az Arcade Mania frontendje több különálló komponensből épül fel, amelyek egymással együttműködve alkotják a teljes felhasználói felületet.

A komponensek egyik fontos eleme a Header komponens, amely az alkalmazás navigációját biztosítja. A Header tartalmazza az alkalmazás címét, a menüt, valamint a különböző navigációs elemeket.

A komponens dinamikusan változik attól függően, hogy a felhasználó be van-e jelentkezve. Ha a felhasználó nincs bejelentkezve, akkor a rendszer a login vagy register gombokat jeleníti meg. Ha a felhasználó be van jelentkezve, akkor a menüben megjelennek a profil és ranglista opciók.

A komponensek közötti adatátadás React state és props segítségével történik. Ez lehetővé teszi, hogy az alkalmazás különböző részei szinkronban maradjanak egymással.

A komponens alapú felépítés nagy előnye, hogy a kód moduláris és könnyen bővíthető. Új funkciók hozzáadása esetén elegendő egy új komponenst létrehozni, amely a meglévő struktúrába illeszthető.

Játékok implementációja

A rendszer egyik legfontosabb része a játékok implementációja. A frontend jelenleg három különböző játékot tartalmaz:

- Fighter
- Memory
- Snake (7.ábra)

7. ábra – Snake game

A Fighter játék egy klasszikus arcade stílusú lövöldözős játék, amelyben a játékos egy űrhajót irányít, és ellenségeket kell kilőnie. A játék HTML Canvas segítségével lett megvalósítva.

A játék motorja JavaScript alapokon működik, és folyamatos frissítési ciklust használ a grafika megjelenítésére. A játék logikája például az ellenségek mozgását, az ütközések felismerését és a pontszám számítását kezeli.

A Memory játék egy kártyapárosító játék, amelyben a játékosnak azonos kártyapárokat kell megtalálnia. A játék során a rendszer számolja a lépéseket, és a játék végén eltárolja az eredményt.

A Snake játék egy klasszikus arcade játék, amelyben a játékos egy folyamatosan növekvő kígyót irányít.

A játékok végén a frontend elküldi az elért pontszámot a backend API számára, amely eltárolja az adatot az adatbázisban.

API kommunikáció a frontendben

A frontend alkalmazás külön szolgáltatásréteget használ az API kommunikáció kezelésére. Ez a réteg a services mappában található JavaScript fájlokból áll.

A legfontosabb szolgáltatások a következők:

- authService
- userService
- gameService

Az authService kezeli a felhasználói hitelesítéssel kapcsolatos műveleteket, például a bejelentkezést és a token kezelését.

A userService a felhasználói adatok lekérdezéséért felel.

A gameService kezeli a játékokkal kapcsolatos API hívásokat, például a pontszámok feltöltését vagy lekérdezését.

Ez a szolgáltatásréteg segít abban, hogy az API kommunikáció elkülönüljön a felhasználói felület logikájától. Ennek köszönhetően az alkalmazás kódja átláthatóbb és könnyebben karbantartható.

Állapotkezelés és állapotlogika

A React alapú alkalmazások egyik fontos eleme az állapotkezelés (state management). Az Arcade Mania frontend alkalmazásában az állapotkezelés elsősorban React state és React Context segítségével történik.

A globális állapot kezelésére a rendszer React Context megoldást használ. Ez különösen fontos az autentikáció kezelésében. Az AuthProvider komponens biztosít egy központi kontextust, amely tartalmazza a bejelentkezett felhasználó adatait, valamint az autentikációval kapcsolatos műveleteket.

Ennek a megoldásnak az az előnye, hogy a felhasználói adatok az alkalmazás bármely pontjáról elérhetők anélkül, hogy az adatokat minden komponensen keresztül külön át kellene adni. Ez egyszerűbbé és átláthatóbbá teszi az alkalmazás felépítését.

A játékok esetében az állapotkezelés különösen fontos szerepet játszik. A játék komponensek folyamatosan frissítik az állapotukat a játék során. Ilyen állapot lehet például a pontszám, a játék vége, vagy a játék aktuális állapota. Ezek az adatok határozzák meg, hogy a felhasználói felület milyen információkat jelenít meg.

Az állapotkezelés segítségével a React automatikusan frissíti a felhasználói felületet, amikor az alkalmazás állapota megváltozik. Ez biztosítja, hogy a felhasználó mindig a legfrissebb adatokat lássa.

Tesztelés

A projekt komponenseinek tesztelése

Egy szoftverrendszer fejlesztése során kiemelten fontos a tesztelés, amely biztosítja, hogy az alkalmazás megbízhatóan és a tervezett módon működjön. Az Arcade Mania projekt esetében a tesztelés több különböző szinten történt, hogy a rendszer különböző komponensei megfelelően működjenek.

A tesztelés során ellenőrzésre került a frontend felület működése, a backend API végpontok helyes működése, a WPF admin felület, valamint a kliens és szerver közötti kommunikáció is. A tesztelési folyamat célja az volt, hogy a rendszer minden fontos funkciója stabilan működjön, és a felhasználók számára hibamentes élményt biztosítson.

A frontend alkalmazás tesztelése során egységtesztek készültek a React komponensek működésének ellenőrzésére. Ezek a tesztek azt vizsgálják, hogy a felhasználói felület megfelelően reagál-e a különböző bemeneti adatokra és eseményekre. A frontend tesztek Node.js környezetben futottak, és a Vitest tesztkeretrendszert használták.

A backend rendszer tesztelése során integrációs tesztek készültek, amelyek az API végpontok működését ellenőrizték. Ezek a tesztek például a felhasználói regisztrációt, a bejelentkezési folyamatot és a pontszámok kezelését vizsgálták. A backend tesztek C# nyelven készültek, és a Microsoft MSTest keretrendszert használták.

A rendszer működésének ellenőrzésére funkcionális tesztek is készültek, amelyek valós felhasználói műveleteket szimulálnak. Ezt a tesztet Selenium WebDriver segítségével került megvalósításra, amely lehetővé teszi a böngésző automatikus vezérlését.

Összegzés

A projekt összegzése

Az Arcade Mania projekt egy full-stack webalkalmazás megvalósítását tűzte ki célul, amelyben a frontend, a backend és az adatbázis rendszer egy egységes platformot alkot. A projekt során egy olyan online arcade játékplatform készült el, amely lehetővé teszi a felhasználók számára különböző játékok kipróbálását, valamint az elért eredmények rögzítését és összehasonlítását.

A rendszer React alapú frontend felületből, ASP.NET Core Web API backendből, a WPF admin felületből és MySQL adatbázisból épül fel. A különböző komponensek REST API segítségével kommunikálnak egymással, ami jól elkülöníti a kliens és szerver oldali logikát. Ez a megközelítés modern és széles körben alkalmazott architektúrát képvisel a webfejlesztésben.

A fejlesztés során több fontos technológiai megoldás került alkalmazásra. A frontend React komponens alapú felépítése lehetővé tette a felhasználói felület moduláris kialakítását, míg a backend ASP.NET Core keretrendszere stabil és jól skálázható szerveroldali infrastruktúrát biztosított. Az Entity Framework Core ORM réteg megkönnyítette az adatbázis műveletek kezelését.

A fejlesztési folyamat során a csapat tagjai különböző területeken dolgoztak, azonban a rendszer működéséhez folyamatos együttműködésre volt szükség. A backend, frontend és adatbázis komponensek integrációja során számos technikai kihívással kellett szembenézni, amelyek megoldása jelentős szakmai tapasztalatot biztosított.

A projekt elkészítése során nemcsak egy működő alkalmazás jött létre, hanem a fejlesztésben részt vevő csapattagok értékes gyakorlati tapasztalatot is szereztek a modern webfejlesztési technológiák használatában. A fejlesztési folyamat során szerzett tudás hozzájárul a későbbi szakmai fejlődéshez is.

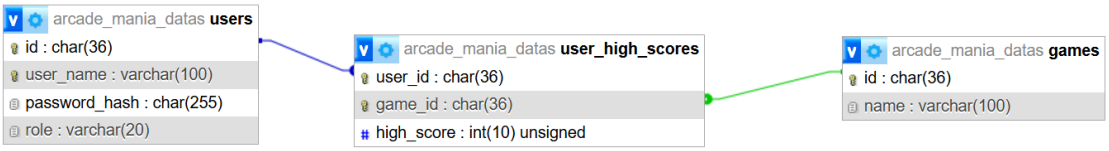
Összességében az Arcade Mania projekt sikeresen bemutatja egy komplex webalkalmazás teljes felépítését, a frontend és backend rendszerek együttműködését, valamint az adatkezelés és autentikáció gyakorlati megvalósítását. A rendszer megfelelő alapot biztosít további fejlesztésekhez is, például új játékok hozzáadásához vagy a platform funkcionalitásának bővítéséhez.

Irodalomjegyzék

1. Téma: React JavaScript könyvtár dokumentációja, Webcím: https://react.dev , letöltés dátuma: 2025.10.15.
2. Téma: ASP.NET Core Web API fejlesztés, Webcím: https://learn.microsoft.com/en-us/aspnet/core/web-api , letöltés dátuma: 2025.10.14.
3. Téma: Entity Framework Core ORM dokumentáció, Webcím: https://learn.microsoft.com/en-us/ef/core , letöltés dátuma: 2026.03.18.
4. Téma: MySQL adatbázis-kezelő dokumentáció, Webcím: https://dev.mysql.com/doc , letöltés dátuma: 2025.10.14.
5. Téma: JSON Web Token (JWT) autentikáció működése, Webcím: https://jwt.io/introduction , letöltés dátuma: 2025.12.05.
6. Téma: React Router navigációs rendszer dokumentációja, Webcím: https://reactrouter.com , letöltés dátuma: 2025.12.10.
7. Téma: Selenium WebDriver tesztelési keretrendszer dokumentációja, Webcím: https://www.selenium.dev/documentation , letöltés dátuma: 2025.09.23.
8. Téma: Windows Presentation Foundation (WPF) dokumentáció, Webcím: https://learn.microsoft.com/en-us/dotnet/desktop/wpf , letöltés dátuma: 2025.11.17.
9. Téma: REST API architektúra és működés, Webcím: https://restfulapi.net , letöltés dátuma: 2025.10.28.
10. Téma: HTTP protokoll működése, Webcím: https://developer.mozilla.org/en-US/docs/Web/HTTP , letöltés dátuma: 2025.11.12

Melléklet

Adatbázis modell:



Frontend komponens példa (RequireAuth):

```
export function RequireAuth({ children }) {
  const auth = useAuth()
  const location = useLocation()

  if (!auth || !auth.user) {
    return <Navigate to="/login" state={{ from: location }} replace />
  }

  return children
}
```

Backend API példa (Login):

```
//  Admin login
[AllowAnonymous]
[HttpPost("admin/login")]
2 references
public async Task<ActionResult> AdminLogin(UserLoginDto dto)
{

    try
    {

        var user = await _context.Users.FirstOrDefaultAsync(u => u.UserName == dto.Name);

        if (user == null)
        {

            return StatusCode(401, new { message = "Hibás felhasználónév vagy jelszó.", result = "" });

        }

        string storedPlainPassword;

        try
        {

            storedPlainPassword = EncryptProvider.AESDecrypt(user.PasswordHash, _passwordKey);

        }
        catch
        {

            return StatusCode(401, new { message = "Hibás felhasználónév vagy jelszó.", result = "" });

        }

        if (storedPlainPassword != dto.Password)
        {

            return StatusCode(401, new { message = "Hibás felhasználónév vagy jelszó.", result = "" });

        }

        var role = string.IsNullOrWhiteSpace(user.Role) ? "User" : user.Role;


        if (!string.Equals(role, "Admin", StringComparison.OrdinalIgnoreCase))
        {

            return StatusCode(403, new { message = "Nincs admin jogosultság.", result = "" });

        }

        var token = _jwtService.GenerateJwtToken(
            subjectId: user.Id.ToString(),
            name: user.UserName,
            role: "Admin"
        );

        var result = new
        {
            Id = user.Id,
            Name = user.UserName
        };

        return StatusCode(200, new { message = "Sikeres admin bejelentkezés", token, result });
    }
    catch (Exception ex)
    {

        return StatusCode(400, new { message = ex.Message, result = "" });

    }
}
```